SIMATS ENGINEERING
ASSESSMENT TOOLS

COURSE NAME: Theory of Computation **COURSE CODE:** CSA13

COURSE OUTCOME COVERED

CO5: *Design Pushdown Automata and analyze their equivalence with Context-Free Grammars through formal construction and proof techniques. (BL: 3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:25**
Duration : 1 Hour

Design Critique Session

Code of Conduct:

Shaik Hafeeza Tasanmym

I 192372044 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Hafeeza

SIMATS ENGINEERING
ASSESSMENT TOOLS

1 A PDA is designed to recognize a^nb^n, but it fails for a^nb^nC^n. Critique the design and identify possible flaws in **stack operations and state transitions**. How would you improve it?

2 A student claims that **Context-Free Grammars (CFGs)**

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid \mathbf{id}$

can be directly converted into a Deterministic PDA (DPDA).Critically evaluate this statement with justification and suggest corrections if needed

3 A Turing Machine is designed using **multiple tracks** to solve a problem. Suggest alternative **programming techniques** like checking-off symbols or subroutines to do subtraction

4 While designing a PDA accepts the language

$$L = \{w \mid w \in (a+b)* \text{ and } n_a(w) == n_b(w)\}$$

$n_a(w)$ is the number of a's in w

$n_b(w)$ is the number of b's in w, acceptance is defined only by **final state**, ignoring empty stack acceptance. Critique this design choice. Would it affect the language recognized? Explain with reasoning.

5 A system designer suggests replacing a Turing Machine with a PDA to reduce computational complexity. Critically compare **FA, PDA, and Turing Machine** capabilities and evaluate whether this replacement is valid.

Critique of PDA designed for $a^n b^n$ but failing for $a^n b^n c^n$

→ A PDA designed for $L = \{a^n b^n\}$ usually pushes one stack symbol for every a and pops one symbol for every b. It cannot directly handle c's.

Possible flaws:

- The PDA may have no transition defined for input symbol c.
- The stack may have become empty after processing all b's, leaving no information to compare the number of c's.
- The state transitions may move directly to an accepting state after perseing b's.

Improvements:

To recognize $a^n b^n c^n$, a single-stack PDA is not sufficient because this language is not Context Free. A Turing machine (or) a more powerful computational model is required.

$a^n b^n c^n$ cannot be recognized by a standard PDA

Can the given CFG be directly converted into a DPDA?

Given $E \to E+T \mid T$

$T \to T*F \mid F$

$F \to (E) \mid id.$

The student's statement is not completely correct. A general CFG can always be converted into a non-deterministic PDA (NPDA). But not every CFG can be directly converted into a deterministic PDA (DPDA).

Justification:

The given grammar contains left recursion

$E \rightarrow E+T$

$T \rightarrow T*F$

left recursion causes difficulty in direct deterministic parsing

Correction:

The grammar should first be transformed into an equivalent suitable grammar, for example by eliminating left recursion. Then a deterministic parsing technique such as LL(0)LR parsing can be used if the resulting

Thus:

> CFG → NPDA is always possible, but CFG → DPDA is not always directly possible.

3. Alternatives to multiple Tracks in a Turing machine.

Instead of using multiple tracks, the Turing machine can use alternative programming techniques.

Checking-off symbols:

symbols that have already been processed can be replaced with marked symbols such as:

$0 \rightarrow X$

$1 \rightarrow Y$.

This helps the machine remember which symbols have already been processed

Subroutines:

The Turing machine can divide a complex operation into smaller tasks. For subtraction, subordinates can perform operations such as:

1. Locate a digit
2. Subtract the corresponding value

3. perform borrowing if necessary
4. Return to next digit

PDA Acceptance by Final State vs Empty stack.

The language is.

$$L = \{w \mid w \in \{a,b\}^*, n_a(w) = n_b(w)\}$$

A PDA may accept either.

- By final state, or
- By Empty stack

For non-deterministic PDAs, these two acceptance methods are Equivalent in computational power. Therefore, using only final-state acceptance does not necessarily change the language recognized.

Reason:

The PDA must ensure that it enters the final state only after all unmatched a's and b's have been properly matched.

If the PDA enters the final state without checking the stack condition, it may incorrectly accept strings with unequal numbers of a's and b's.

Thus,

> Final-state acceptance is valid, but proper stack verification is necessary.

(5) Comparison of FA, PDA and Turing Machine.

Model	Memory	Language Capability
FA	Finite Memory	Regular Languages.
PDA	Stack memory	Context-Free languages
Turing machine	Unlimited tape	Recursively Enumerable Languages.

Critical Evaluation:

⇒ A PDA is more powerful than a Finite Automation but less powerful than a Turing machine

⇒ A system designer cannot always replace a Turing machine with a PDA merely to reduce computational complexity.

⇒ A PDA has only one stack and cannot solve problems requiring unlimited two-way tape acess.

$$L = \{ a^n b^n c^n \mid n \geq 1 \}$$

Cannot be recognized by a standard PDA but can be recognized by Turing machine

Conclusion:

Replacing a Turing machine with a PDA is valid only when the problem belongs to the class of context-free languages.

TM > PDA > FA

SIMATS ENGINEERING
ASSESSMENT TOOLS

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

89/100